```
std::condition_variable cond;
bool data_ready;
std::mutex m;

void wait_for_data()
{
    std::unique_lock<std::mutex> lk(m);
    cond.wait(lk,[]{return data_ready;});     <-- (1)
}
```

The return type of the lambda passed to `cond.wait()` (1) is deduced from the type of `data_ready` and is thus `bool`. Whenever the condition variable wakes from waiting, it then calls the lambda with the mutex locked and only returns from the call to `wait()` once `data_ready` is `true`.

What if you can't write your lambda body as a single `return` statement? In that case you have to specify the return type explicitly. You can do this even if your body is a single `return` statement, but you *have* to do it if your lambda body is more complex. The return type is specified by following the lambda parameter list with an arrow (`->`) and the return type. If your lambda doesn't take any parameters, you must still include the (empty) parameter list in order to specify the return value explicitly. Your condition variable predicate can thus be written

```
cond.wait(lk,[]()->bool{return data_ready;});
```

By specifying the return type, you can expand the lambda to log messages or do some more complex processing:

```
cond.wait(lk,[]()->bool{
    if(data_ready)
    {
        std::cout<<"Data ready"<<std::endl;
        return true;
    }
    else
    {
        std::cout<<"Data not ready, resuming wait"<<std::endl;
        return false;
    }
});
```